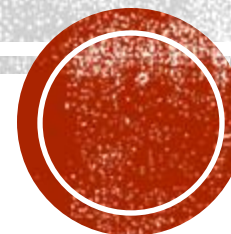


WEEK 10

Keras Library

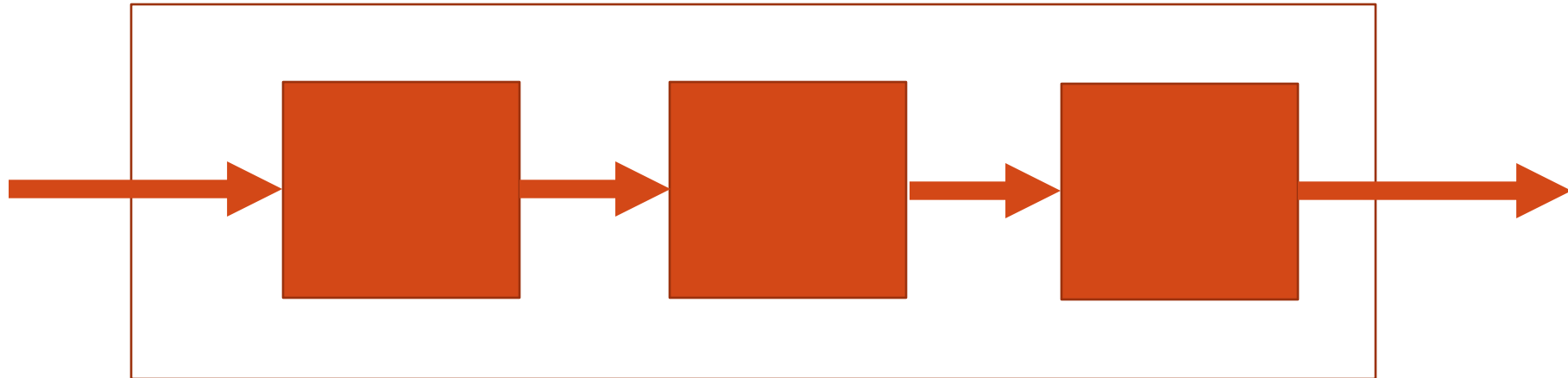


TENSORFLOW

- Using Keras library
- Pip install tensorflow
- Pip install keras



SEQUENTIAL MODEL



USING KERAS WITH REGRESSION PROBLEMS

```
import keras  
  
from keras.models import Sequential  
from keras.layers import Dense  
  
import numpy as np  
  
trX = np.linspace(-1, 1, 101)  
trY = 3 * trX + np.random.randn(*trX.shape) * 0.33
```

[Keras.ipynb](#)



CREATE LINEAR NETWORK MODEL

- **To create a model, initialise model ready to create a network**

```
model = Sequential()
```

- **Create a fully connected single layer network**

```
model.add(Dense(1, ' , activation='linear'))
```

- Number = hidden nodes
- activation='linear' create a linear activation function



CREATE LINEAR NETWORK MODEL

```
# Display weight and bias from the input and output layer
```

```
weights = model.layers[0].get_weights()
```

```
w_init = weights[0]
```

```
b_init = weights[1]
```

```
print('Linear regression model is initialized with weights w: %.2f, b: %.2f' %  
(w_init, b_init))
```



COMPILE AND FIT

- **Set how each parameter is set for training**

```
model.compile(optimizer='sgd', loss='mse')
```

- **Fit the model with max. epoch 200**

```
model.fit(trX, trY, epoch=200, verbose=1)
```

- **Show weight and biased after fitting the model**

```
weights = model.layers[0].get_weights()
```

- **Show the model after training**

```
w_final = weights[0]
```

```
b_final = weights[1]
```

```
print('Linear regression model is trained to have weight w: %.2f, b: %.2f' % (w_final, b_final))
```



PLOT

```
import matplotlib.pyplot as plt  
plt.plot(trX,trY,'ro')  
plt.plot(trX,trX*w_final+b_final)  
plt.show()
```



SAVE AND LOAD MODEL

- `model.save_weights("my_model.h5")`
- `model.load_weights('my_model_weights.h5')`

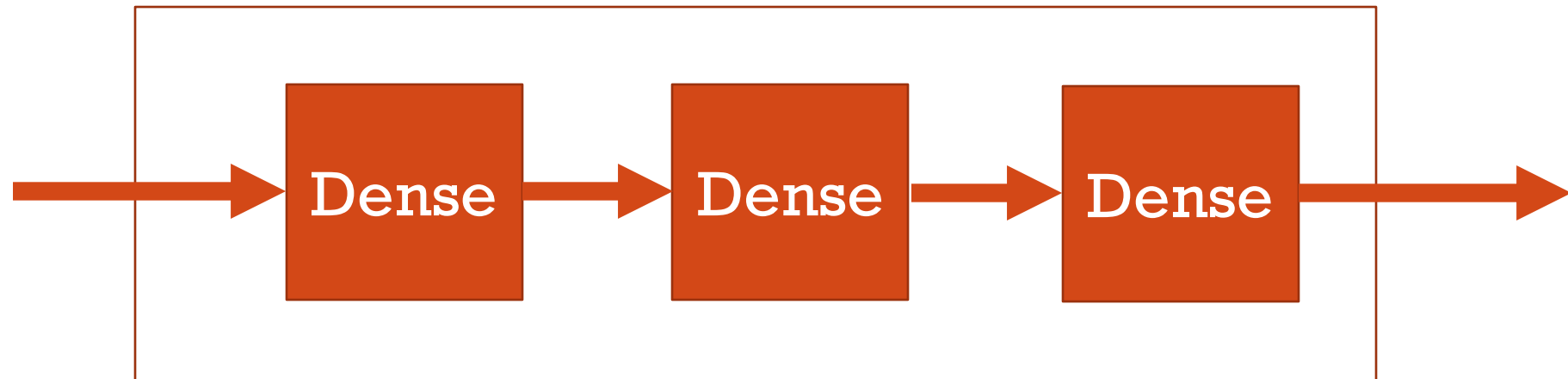


DROPOUT

- To prevent Neural network from over fitting
- Range of dropout between 0 to 1



MLP



BINARY OUTPUT

- Output layer
 - 1 output
 - Sigmoid
- Evaluation/loss
 - `binary_crossentropy`
- `model.add(Dense,1)`



CROSS ENTROPY: BINARY

Sum-of-squares is more suitable for regression because of the target

- More appropriate error estimation of classification

For network with a single output, representing 2 classes

- Probability to be class 1 $y=P(C_1|x)$

$$E = - \sum \left\{ t \ln \frac{y}{t} + (1 - t) \ln \frac{(1 - y)}{(1 - t)} \right\}$$

- Probability to be class 2 $P(C_2|x)=1-y$
- t is a target coding, $t=1$ means its belong to class 1 and 0 otherwise



MULTIPLE OUTPUT

- Output layer
 - 1-of-c class
 - Softmax
- Evaluation
 - `catergorical_crossentropy`



CROSS ENTROPY: MULTICLASS(CATEGORICAL)

For recognising c patterns, using 1-of- c target scheme

- c outputs for c patterns/classes

The cross entropy error is:

$$E = - \sum \sum_{k=1}^c \left\{ t_c \ln \frac{y_c}{t_c} + (1 - t_c) \ln \frac{(1 - y_c)}{(1 - t_c)} \right\}$$

Summing the cross entropy across all network outputs or prob. outputs together with number of data.



MNIST EXAMPLE: NEURAL NETWORK

- `y_test = keras.utils.to_categorical(Y_test, num_classes=10)`

- To create 10 output from 1

- 1 → [0 1 0 0 0 0 0 0 0 0]

- 9 → [0 0 0 0 0 0 0 0 0 1]

- [KERAS2.ipynb](#)



SOME OTHER TYPES OF NEURAL NETWORKS

- RBF Neural Network
 - Use Radial Basis function as activation function
 - Good for continuous output
- RNN Network
 - Each hidden node receives its own value as input
 - Good for predicting text, the next word uses the current word to analysed
- Auto Encoder
 - Unsupervised Neural network



SOME OTHER TYPES OF NEURAL NETWORKS

- Kohonen Network (Self Organizing Map Network)
 - Use a map in hidden layer(competition layer) to identify the real data
 - It has shown
- Convolution Neural Network
 - Divide input features in batches.
 - Use extensively in image recognition
- Modular Neural Network
 - Break down network into different modules
 - Use for decision making

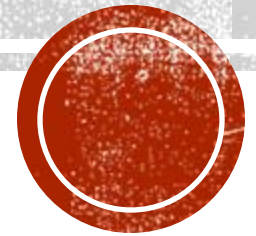


REFERENCES

- <http://iopscience.iop.org/article/10.1088/1742-6596/312/6/062005/pdf>
- <https://www.hindawi.com/journals/cin/2012/654895/>
- <https://www.ncbi.nlm.nih.gov/pubmed/9481716>
- <https://arxiv.org/pdf/1702.07825.pdf>
- <http://www.worldscientific.com/doi/abs/10.1142/S0129065799000125>



TUNING



TYPICAL PREPOSSESSING

Normal Dataset

- Zero mean/ unit variance `preprocessing.StandardScaler()`
- Image Dataset
- Rescale 0-1 : `data/255`
- Thresholding (`>threshold = 1`)
- `keras.preprocessing.image.ImageDataGenerator`



REDUCE OVER FITTING(REGULARIZATION)

Weight regularization/weight decay

- A penalty add to a loss function (cost function)
- Avoid weight to grow too quickly (put in each layer)

```
kernel_regularizer=regularizers.l2(0.01)
```

- The bigger value the bigger regularization



REDUCE OVER FITTING(DROPOUT)

In each batch, a proportional of units in 1 or more layers are multiplied by 0

- Different batches face different network architecture
- It helps overfitting



REDUCE OVER FITTING(EARLY STOPING)

Set callback functions to early stop training and save the best model so far

```
es = EarlyStopping(monitor='val_loss', patience=2)  
network.fit(...callbacks=es,...)
```

- Save the best model based on val data



BATCH NORMALIZATION

- Batch normalization is applied to individual **layers**
- Avoid over fitting usually for deep network
- Make training faster, Allows higher learning rates
- After compute activation as usual, it normalized activations of each node by subtracting its mean and dividing by its standard deviation

```
model.add(BatchNormalization() )
```



CALLBACKS

- EarlyStopping
- ModelCheckpoints

```
callbacks = [EarlyStopping(monitor='val_loss', patience=2),  
             ModelCheckpoint(filepath='best_model.h5', monitor='val_loss',  
                             save_best_only=True)]  
network.fit(...callbacks=callbacks,...)
```



EXERCISE / LAB

- [KERAS2-with GridSearch.ipynb](#)

- Look at the saved file

- Check the summary

📖 Where do shapes/ parameter are from

📖 Show me the calculation

